

Incremental AI Agent Workflow for Building the Real Human Proof App

Executive summary

Your target stack is well aligned with the current official guidance for modern full-stack web apps: Next.js App Router and Route Handlers for the web application shell and backend-for-frontend layer, Tailwind CSS for styling, Prisma ORM with PostgreSQL for typed data and migrations, Stripe Checkout for one-time and subscription payments, Resend for transactional email from App Router routes, OpenAI's Responses and Moderation APIs for structured AI helpers and safety, and Vercel for deployment. Next.js now ships a first-party PWA guide for App Router, supports version-matched in-package docs for coding agents through `AGENTS.md`, and recommends a Data Access Layer pattern for new projects. Prisma's current docs explicitly cover Next.js, production migrations, and per-request client extensions that can be used with PostgreSQL row-level security. ¹

For this specific product, the safest MVP is a **privacy-first, capability-tokenized workflow**: authenticated request creators, reply-without-account via signed one-time reply links, opt-in one-degree forwarding, no contact scraping, no public ratings, and a separate moderation/admin surface. PostgreSQL RLS should be treated as a real database boundary, not just app-layer filtering: PostgreSQL's default-deny behavior when RLS is enabled but no policy applies is especially valuable here, and Prisma's per-request client extension pattern is the cleanest official starting point when you want each request to carry its own security context. ²

The most practical delivery plan for a solo founder is **seven incremental sprints** over roughly **five to eight weeks** of focused work with an AI coding agent. Ship the core proof-request flow first, then add reply-without-account, evidence uploads, forwarding, payments, AI request improvement, moderation, and only then PWA polish, analytics, observability, and stricter security hardening. If you keep every agent task to one vertical slice with tests, migrations, and deploy notes, the tooling will stay manageable and the codebase will remain reviewable. This sequencing is a product recommendation, but it is grounded in the way the official tools are designed to be adopted incrementally. ³

Reference architecture

The application should be organized as a **single Next.js 16 App Router monolith** with four clear zones: the public marketing/PWA shell, authenticated founder/admin and creator dashboards, public capability-token entrypoints for reply and forwarding, and a server-only Data Access Layer. Next.js recommends a Data Access Layer approach for new projects, and Route Handlers are the natural fit for tokenized public endpoints because they use the standard Web `Request` and `Response` APIs inside the `app` directory. ⁴

Use **Prisma 7 + PostgreSQL** as the source of truth, but do not rely on Prisma alone for authorization. Enforce authorization twice: first in application code through a server-only DAL, then in PostgreSQL with RLS policies. PostgreSQL's RLS can restrict visibility and write access per row and per command, and if RLS is enabled with no applicable policy, the database falls back to default-deny. Prisma's client extension model is explicitly documented as a way to create a request-scoped client for RLS-style isolation. In practice, that means every authenticated request or public capability token should set a database context value before queries run. 5

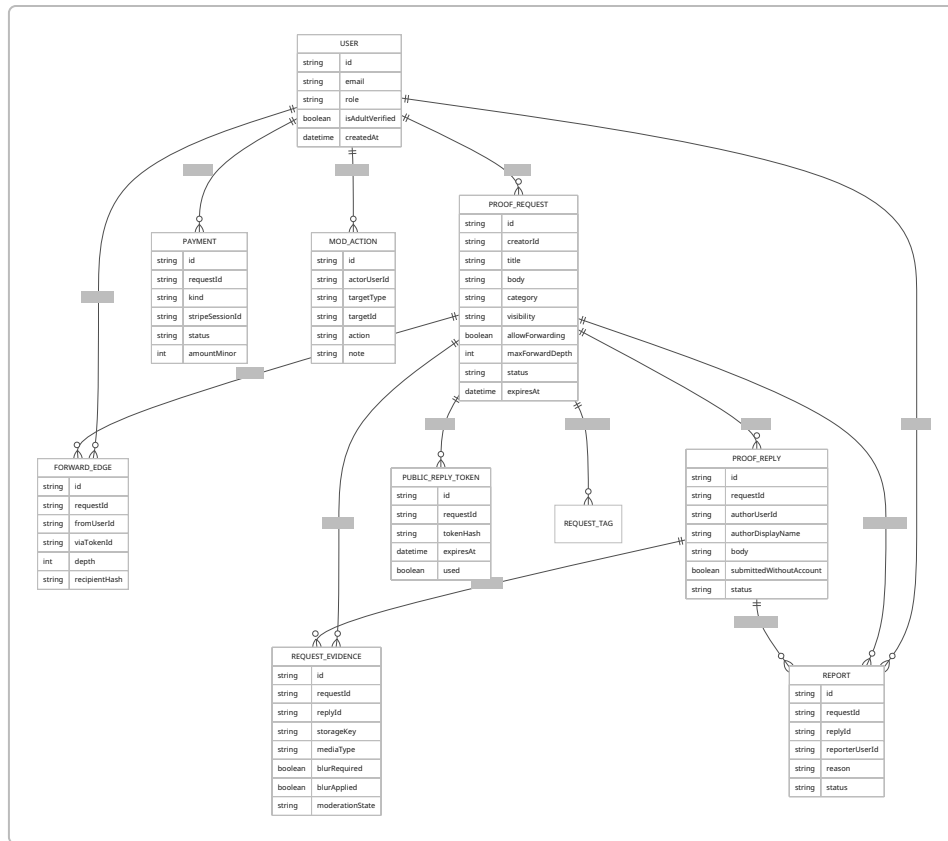
For **uploads**, pick one path and commit to it for MVP. If speed matters most, UploadThing is the fastest fit because its App Router setup is native, its FileRouter can constrain file type/count/size, and it supports route-level middleware and validation. If long-term storage portability and lower-level control matter more, Cloudflare R2 is a better fit because it exposes S3-compatible object storage and presigned URLs for direct uploads without exposing API credentials. My recommendation for the first build is **UploadThing for MVP, R2 later if you need storage portability or your own media pipeline workers**. That is an architectural inference from the official docs, not a vendor claim. 6

For **payments**, keep phase one simple: platform donations, urgent boosts, and feature unlocks through Stripe Checkout. Stripe's Checkout Sessions API supports one-time and subscription payments and a very broad set of payment methods, which is enough for your initial monetization model. Defer helper payouts until later; once you start moving money between multiple parties, you are in Stripe Connect / marketplace territory, which adds onboarding, payout, risk, and compliance work. 7

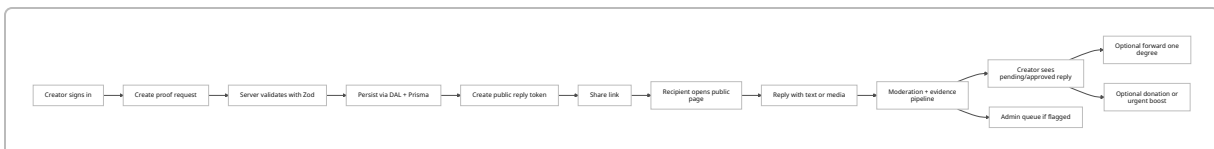
For **AI**, use OpenAI in three bounded roles: text improvement and translation through the Responses API, strongly typed AI outputs through Structured Outputs, and moderation on both text and images through the Moderation API. Do **not** make OpenAI the canonical decision-maker for account bans or legality; use it as triage into queues, trust scoring, and human review. For evidence redaction, use OpenAI for analysis/triage if useful, but do the actual blur in a deterministic media-processing step you control. OpenAI's docs support text generation, structured JSON outputs, image analysis, and moderation, but they do not turn storage providers into privacy-redaction systems. 8

The core data model should separate **private evidence**, **public capability tokens**, and **moderation artifacts** so you can keep the default UX private-by-default and still allow low-friction replies. That separation also aligns with GDPR's "data protection by design and by default," especially the requirement that personal data not be made accessible by default to an indefinite number of people. 9

The following model is a strong starting point for the MVP architecture:



The request flow should keep public access narrow and revocable:



Sprint roadmap

The table below gives you the recommended order, solo-founder estimate, and priority. The time estimates assume you are using an AI coding agent aggressively but still reviewing all code yourself.

Sprint	Scope	Solo founder estimate	Priority
Foundation	Bootstrap repo, AGENTS.md rules, Tailwind, Prisma, auth skeleton, DAL, test setup, env schema	2-3 days	Critical
Data security	Core Prisma schema, initial migrations, RLS SQL, request-scoped Prisma client, role model	2-4 days	Critical

Sprint	Scope	Solo founder estimate	Priority
Core request flow	Request creation UI, creator dashboard, public request page, share links	3–5 days	Critical
Replies and evidence	Reply-without-account, UploadThing or R2 upload flow, moderation queue, blur pipeline shell	4–6 days	Critical
Forwarding and notifications	One-degree forwarding, expiring forward tokens, Resend email notifications	2–4 days	High
Payments and AI helpers	Stripe Checkout, donations/boosts, AI request improver, translation, structured outputs	3–5 days	High
Moderation and launch hardening	Admin panel, reports/blocks, audit log, analytics, Sentry, PWA, CI/CD, Vercel deploy	4–7 days	Critical

This order reflects two constraints from the official docs and the product risk profile: database migrations should run in CI/CD rather than ad hoc from your laptop, and security boundaries should exist before growth features and analytics. Prisma’s production guidance recommends `migrate deploy` in an automated pipeline, while Next.js emphasizes explicit data-security thinking and a DAL for new projects.

10

Copy paste prompts and implementation sequence

Sprint foundation

Start by generating a **reviewable skeleton**, not features. Use `create-next-app` with TypeScript, Tailwind, App Router, and AGENTS support. Next.js now includes `AGENTS.md` and `CLAUDE.md` by default, and those files are specifically meant to steer coding agents toward version-matched docs inside the installed `next` package. Tailwind’s current Next.js setup uses `@tailwindcss/postcss`, and Prisma’s Next.js guide documents a clean initial install flow for Prisma ORM with PostgreSQL. ¹¹

```
pnpm create next-app real-human-proof --ts --tailwind --eslint --app --
turbo --use-pnpm --yes
cd real-human-proof

pnpm add @prisma/client @prisma/adapter-pg pg zod stripe resend posthog-js
@sentry/nextjs openai lucide-react
pnpm add -D prisma tsx vitest @vitest/ui @testing-library/react @testing-
library/jest-dom @playwright/test

pnpm add uploadthing @uploadthing/react
pnpm dlx prisma init --output ../app/generated/prisma
```

Use this agent prompt:

Read AGENTS.md and the version-matched Next.js docs before making changes.

Task: set up the project skeleton for a production-oriented Next.js 16 App Router PWA called "real-human-proof".

Requirements:

- Keep App Router.
- Keep TypeScript strict mode.
- Use Tailwind CSS 4.
- Add lucide-react.
- Add Prisma 7 with PostgreSQL driver adapter pattern.
- Add Zod for env and API validation.
- Add Vitest + Testing Library for unit tests.
- Add Playwright for E2E.
- Add a server-only Data Access Layer folder.
- Add a minimal auth abstraction interface, but do not fully implement auth provider specifics yet.
- Add app/manifest.ts, icons placeholders, robots, sitemap, and a basic offline-friendly PWA shell.
- Add an env schema and fail-fast startup validation.
- Add AGENTS.md project-specific rules outside the Next.js-managed markers.

Output:

- working dev server
- working test command
- no product features yet
- concise README with setup commands
- no placeholder secrets committed

After coding:

- run typecheck, lint, unit tests
- show changed files
- summarize risks and TODOs

Expected outputs should include `app/manifest.ts`, `lib/env.ts`, `lib/dal/`, `lib/auth/`, `prisma/schema.prisma`, `vitest.config.ts`, `playwright.config.ts`, and a project-specific `AGENTS.md` section. The PWA shell is justified by the official Next.js PWA guide, which documents manifest support in App Router and a service-worker-based path to installability and notifications. ¹²

Run these checks after the agent completes the sprint:

```
pnpm lint
pnpm exec tsc --noEmit
```

```
pnpm vitest run
pnpm exec playwright install
```

A minimal env schema is worth creating immediately because Vercel environment variables are encrypted at rest, available during build and function execution, and manageable through CLI workflows such as

```
vercel env add, vercel env pull, and vercel env run.
```

13

Sprint data security

This sprint creates the **real security boundary**. Implement users, roles, requests, replies, evidence, payments, reports, and moderation actions. Then add PostgreSQL RLS and create a request-scoped Prisma client extension that sets database context per request. PostgreSQL's `CREATE POLICY`, `USING`, and `WITH CHECK` semantics are exactly what you need for a private-by-default system with narrowly scoped public writes. Prisma's docs explicitly call out RLS and request-specific clients as a supported extension use case.

5

Use this agent prompt:

Task: implement the first real data model and database security layer.

Requirements:

- Create Prisma models for User, ProofRequest, ProofReply, RequestEvidence, PublicReplyToken, ForwardEdge, Payment, Report, ModAction.
- Add enums for role, request status, reply status, payment kind, report status.
- Keep all personally identifying data minimal.
- Add soft-delete and audit timestamps where useful.
- Add a request-scoped Prisma client helper that sets PostgreSQL session config:
app.user_id
app.role
app.reply_token_id
- Add SQL migrations for:
 - enabling row level security
 - SELECT/INSERT/UPDATE policies for creator-owned rows
 - public insert-only policy for valid reply tokens
 - admin override policy
- Add DAL functions only; no direct Prisma import in route handlers or components.
- Add unit tests for DAL authorization behavior.
- Add a seed script with one admin and one normal account.

Important:

- Do not expose raw Prisma client to route handlers.
- Keep policies default-deny unless explicitly allowed.
- Store reply tokens hashed, never plaintext.

A good first Prisma schema nucleus looks like this:

```

enum UserRole {
    USER
    ADMIN
}

enum RequestStatus {
    OPEN
    UNDER_REVIEW
    RESOLVED
    CLOSED
}

enum ReplyStatus {
    PENDING
    APPROVED
    REJECTED
}

model User {
    id          String    @id @default(cuid())
    email       String    @unique
    role        UserRole  @default(USER)
    isAdultVerified Boolean @default(false)
    createdAt   DateTime  @default(now())
    updatedAt   DateTime  @updatedAt

    requests    ProofRequest[]
    reports      Report[]
    modActions   ModAction[]
}

model ProofRequest {
    id          String    @id @default(cuid())
    creatorId   String
    title       String
    body        String
    category    String
    visibility   String    @default("PRIVATE_LINK")
    allowForwarding Boolean @default(true)
    maxForwardDepth Int     @default(1)
    status      RequestStatus @default(OPEN)
    expiresAt   DateTime?
    createdAt   DateTime    @default(now())
    updatedAt   DateTime    @updatedAt

    creator     User        @relation(fields: [creatorId], references: [id])

```

```

    replies          ProofReply[]
    evidence          RequestEvidence[]
    replyTokens       PublicReplyToken[]
    forwardEdges      ForwardEdge[]
    payments           Payment[]
    reports            Report[]
}

```

A representative SQL migration pattern is:

```

ALTER TABLE "ProofRequest" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "ProofReply" ENABLE ROW LEVEL SECURITY;

CREATE POLICY proof_request_owner_select
ON "ProofRequest"
FOR SELECT
USING (
    "creatorId" = current_setting('app.user_id', true)
);

CREATE POLICY proof_request_owner_write
ON "ProofRequest"
FOR UPDATE
USING (
    "creatorId" = current_setting('app.user_id', true)
)
WITH CHECK (
    "creatorId" = current_setting('app.user_id', true)
);

CREATE POLICY proof_reply_public_insert
ON "ProofReply"
FOR INSERT
WITH CHECK (
    current_setting('app.reply_token_id', true) IS NOT NULL
);

CREATE POLICY admin_all_requests
ON "ProofRequest"
AS PERMISSIVE
FOR ALL
USING (current_setting('app.role', true) = 'ADMIN')
WITH CHECK (current_setting('app.role', true) = 'ADMIN');

```

That exact `current_setting(...)` pattern is an implementation recommendation built on PostgreSQL's documented per-row policy model and Prisma's documented request-scoped extension model. ⁵

Run this after the sprint:

```
pnpm prisma migrate dev --name init_security
pnpm prisma generate
pnpm vitest run
pnpm exec tsc --noEmit
```

Sprint core request flow

Now add the **creator workflow**: create request, list requests, inspect a request, and generate a shareable public reply link. Keep this sprint text-only so you can validate the product loop before dealing with media and moderation complexity. Route Handlers are ideal here because they are native App Router request handlers. Next.js also recommends using an auth library rather than inventing a full custom auth stack unless you have a strong reason. ¹⁴

Use this agent prompt:

Task: build the core proof request flow for authenticated creators.

Requirements:

- Implement creator dashboard routes:
 - /dashboard
 - /dashboard/requests
 - /dashboard/requests/new
 - /dashboard/requests/[id]
- Implement Route Handlers:
 - POST /api/requests
 - GET /api/requests/[id]
 - GET /api/requests
- Validate all input with Zod.
- Create a shareable public page:
 - /r/[requestId]/reply/[token]
- Public page may show only the minimum fields needed to reply.
- Keep all data fetching in the DAL.
- Add form-level validation, inline errors, optimistic UI only if safe.
- Add E2E tests for request creation and viewing.

A minimal Route Handler example:

```
// app/api/requests/route.ts
import { NextResponse } from 'next/server'
import { z } from 'zod'
import { createProofRequest } from '@lib/dal/requests'
import { requireSession } from '@lib/auth/session'
```

```

const CreateRequestSchema = z.object({
  title: z.string().min(10).max(140),
  body: z.string().min(30).max(5000),
  category: z.string().min(2).max(50),
  allowForwarding: z.boolean().default(true),
  expiresAt: z.string().datetime().optional(),
})

export async function POST(req: Request) {
  const session = await requireSession()
  const json = await req.json()
  const input = CreateRequestSchema.parse(json)

  const request = await createProofRequest(session, input)
  return NextResponse.json({ request }, { status: 201 })
}

```

A minimal client component example:

```

'use client'

import { useState } from 'react'

export function NewRequestForm() {
  const [pending, setPending] = useState(false)

  async function onSubmit(formData: FormData) {
    setPending(true)
    try {
      const res = await fetch('/api/requests', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          title: formData.get('title'),
          body: formData.get('body'),
          category: formData.get('category'),
          allowForwarding: formData.get('allowForwarding') === 'on',
        }),
      })
      if (!res.ok) throw new Error('Failed to create request')
    } finally {
      setPending(false)
    }
  }
}

```

```

    return (
      <form action={onSubmit} className="space-y-4">
        <input name="title" placeholder="What do you need verified?" className="w-
full rounded border p-3" />
        <textarea name="body" placeholder="Add context, timing, and constraints"
className="w-full rounded border p-3" />
        <input name="category" placeholder="Category" className="w-full rounded
border p-3" />
        <label className="flex items-center gap-2">
          <input type="checkbox" name="allowForwarding" defaultChecked />
          Allow one-degree forwarding
        </label>
        <button disabled={pending} className="rounded px-4 py-2">
          {pending ? 'Creating...' : 'Create request'}
        </button>
      </form>
    )
  }
}

```

Run these checks:

```

pnpm vitest run
pnpm exec playwright test
pnpm lint
pnpm exec tsc --noEmit

```

Sprint replies and evidence

This sprint introduces the highest abuse risk, so keep the implementation conservative. Add **reply-without-account**, media uploads, an evidence-processing state machine, and a moderation gate. UploadThing is particularly ergonomic here because its FileRouter can constrain file types and sizes and supports route-level validation and middleware; if you choose R2 instead, use server-generated presigned URLs and treat storage as transport only. OpenAI's moderation models can classify text and images, and its image-capable models can analyze image inputs, but the application should still own the publish/no-publish rule and the redaction pipeline. ¹⁵

Use this agent prompt:

Task: implement reply-without-account, evidence uploads, and moderation gating.

Requirements:

- Public POST /api/public/replies using a signed public reply token.
- Token must be hashed in DB and verified in constant-time comparison.
- Support text reply plus image/video uploads.
- Use UploadThing App Router integration for MVP.

- Add moderation state machine:
RECEIVED -> SCANNING -> APPROVED or FLAGGED or REJECTED
- Add an evidence-processing job interface:
 - image moderation
 - blur-required detection flag
 - blur-applied flag
 - media metadata extraction
- Do not make replies visible to creator until moderation state is APPROVED or creator has requested "show pending".
- Add creator UI to view pending, approved, and flagged replies separately.
- Add tests for expired tokens, reused tokens, invalid MIME, oversized uploads.

If you choose UploadThing, your upload entry can follow the documented App Router pattern:

```
// app/api/uploadthing/route.ts
import { createRouteHandler } from 'uploadthing/next'
import { ourFileRouter } from './core'

export const { GET, POST } = createRouteHandler({
  router: ourFileRouter,
})
```

And the FileRouter should keep constraints explicit:

```
// app/api/uploadthing/core.ts
import { createUploadthing, type FileRouter } from 'uploadthing/next'

const f = createUploadthing()

export const ourFileRouter = {
  evidenceUploader: f({
    image: { maxFileSize: '8MB', maxFileCount: 4 },
    video: { maxFileSize: '64MB', maxFileCount: 1 },
  })
  .input(/* zod schema here */)
  .middleware(async ({ input }) => {
    return { requestId: input.requestId, tokenId: input.tokenId }
  })
  .onUploadComplete(async ({ metadata, file }) => {
    // persist file metadata, queue moderation
    return { storageKey: file.key }
  }),
} satisfies FileRouter
```

If you choose R2 instead, the official pattern is presigned URLs for time-limited `PutObject` / `GetObject` access without exposing credentials. That is a better fit if you want total control over the upload UX and storage abstraction. ¹⁶

Tests to run:

```
pnpm vitest run
pnpm exec playwright test tests/public-reply.spec.ts
pnpm lint
pnpm exec tsc --noEmit
```

Sprint forwarding and notifications

This is where virality enters safely: **one-degree forwarding only**. Do not import contacts. Do not scrape address books. Forwarding should happen by explicit invite link or by manually entered email/handle, then hashed at rest where possible. That recommendation is a product and privacy decision, but it aligns with GDPR's privacy-by-default principle. For notifications, use Resend directly from an App Router route or server-side service layer. Resend's own Next.js guide shows the App Router route pattern clearly. ¹⁷

Use this agent prompt:

Task: implement one-degree forwarding and notifications.

Requirements:

- Add a ForwardEdge table and DAL methods.
- Enforce maxForwardDepth = 1 in application logic and database checks.
- Support forwarding by generated invite link and by explicitly typed email.
- Never access user address books.
- Hash recipient email before storage when only uniqueness or dedupe is needed.
- Generate expiring forward tokens.
- Add email notifications through Resend for:
 - request created
 - new approved reply
 - request forwarded to you
 - urgent boost purchased
- Add an inbox panel for creators.
- Add tests for forwarding depth > 1 and token expiry.

A minimal notification endpoint/service example:

```
import { Resend } from 'resend'

const resend = new Resend(process.env.RESEND_API_KEY)
```

```
export async function sendForwardEmail(to: string, requestTitle: string,
replyUrl: string) {
  return resend.emails.send({
    from: process.env.EMAIL_FROM!,
    to,
    subject: `You've been asked to help verify: ${requestTitle}`,
    html: `

Someone forwarded a proof request to you.</p><p><a href="${
{replyUrl}}>Open request</a></p>`,
  })
}


```

Tests:

```
pnpm vitest run
pnpm exec playwright test tests/forwarding.spec.ts
```

Sprint payments and AI helpers

Add monetization only after the base loop is usable. Stripe Checkout is enough for donations, urgent boosts, and feature unlocks. Protect your webhook endpoint by verifying the `Stripe-Signature` header against the raw request body; Stripe's docs are explicit that signature verification must use the raw body and the endpoint secret. For AI, use the Responses API for request improvement/translation and Structured Outputs so the agent returns typed JSON your app can trust. ¹⁸

Use this agent prompt:

Task: add payments and bounded AI helpers.

Requirements:

- Add payment kinds:
 - DONATION
 - URGENT_BOOST
 - FEATURE_UNLOCK
- Implement POST `/api/payments/checkout-session`
- Implement POST `/api/webhooks/stripe` with raw-body signature verification
- On successful checkout, persist payment record and apply product effects idempotently
- Add AI request improver:
 - takes draft title/body and returns improved title/body, missing fields, risk flags
- Add AI translation helper:
 - translate request into target language and preserve meaning
- Use Structured Outputs with strict JSON schema

- Add tests for duplicate webhooks and idempotency
- Do not implement helper payouts yet

A minimal Checkout session creator:

```
// app/api/payments/checkout-session/route.ts
import Stripe from 'stripe'
import { NextResponse } from 'next/server'

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

export async function POST(req: Request) {
  const { requestId, kind } = await req.json()

  const session = await stripe.checkout.sessions.create({
    mode: 'payment',
    line_items: [
      {
        price_data: {
          currency: 'usd',
          product_data: { name: `Real Human Proof ${kind}` },
          unit_amount: kind === 'URGENT_BOOST' ? 300 : 100,
        },
        quantity: 1,
      },
    ],
    success_url: `${process.env.APP_URL}/payments/success?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.APP_URL}/payments/cancel`,
    metadata: { requestId, kind },
  })

  return NextResponse.json({ url: session.url })
}
```

A minimal OpenAI structured-output pattern:

```
import OpenAI from 'openai'

const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY })

export async function improveRequest(input: { title: string; body: string }) {
  const response = await client.responses.create({
    model: 'gpt-5.5',
    input: [
```

```

    {
      role: 'user',
      content: [
        {
          type: 'input_text',
          text: `Improve this proof request while preserving meaning: $
{JSON.stringify(input)}`,
        },
      ],
    },
  ],
  text: {
    format: {
      type: 'json_schema',
      name: 'proof_request_improvement',
      schema: {
        type: 'object',
        additionalProperties: false,
        properties: {
          improvedTitle: { type: 'string' },
          improvedBody: { type: 'string' },
          missingFields: { type: 'array', items: { type: 'string' } },
          riskFlags: { type: 'array', items: { type: 'string' } },
        },
        required: ['improvedTitle', 'improvedBody', 'missingFields',
'riskFlags'],
      },
    },
  },
})

  return JSON.parse(response.output_text)
}

```

For helper payouts, wait. Stripe Connect is the right tool once you are paying third parties or splitting funds, but that turns the product into a platform/marketplace from a payments perspective. ¹⁹

Tests:

```

pnpm vitest run
pnpm exec playwright test tests/payments.spec.ts
stripe listen --forward-to localhost:3000/api/webhooks/stripe

```

Sprint moderation and launch hardening

Now build the **minimal admin moderation panel**, hook up Sentry and PostHog carefully, and finish the PWA and deployment path. The admin panel should be intentionally boring: queue, detail view, approve/reject, block requester/replier, audit note, and immutable moderation log. The DSA requires easy flagging/reporting and user recourse around moderation decisions, so at minimum your public/app UX should expose report actions, a contact point, and a traceable moderation reason. 20

Use this agent prompt:

Task: build moderation, admin tooling, observability, and release hardening.

Requirements:

- Add /admin protected routes for ADMIN role only.
- Build moderation queue pages:
 - /admin
 - /admin/reports
 - /admin/replies/[id]
 - /admin/requests/[id]
- Add actions:
 - approve reply
 - reject reply
 - require blur
 - suspend token
 - block user
 - close request
- Persist every admin action in ModAction with actor, target, action, note.
- Add Sentry for client/server/edge.
- Add PostHog analytics with a reverse proxy path and no session recordings by default.
- Add CSP headers and tighten image/media origins.
- Finish manifest/icons/service worker registration.
- Add E2E coverage for report flow and admin action flow.
- Output a release notes document and deployment checklist.

A minimal admin queue component:

```
import { listOpenReports } from '@lib/dal/admin'

export default async function AdminReportsPage() {
  const reports = await listOpenReports()
  return (
    <main className="mx-auto max-w-5xl p-6">
      <h1 className="text-2xl font-semibold">Open reports</h1>
      <ul className="mt-6 space-y-3">
```

```

    {reports.map((report) => (
      <li key={report.id} className="rounded border p-4">
        <div className="font-medium">{report.reason}</div>
        <div className="text-sm text-muted-foreground">
          Target: {report.targetType} 🔴 Status: {report.status}
        </div>
      </li>
    ))}
  </ul>
</main>
)
}

```

For **Sentry**, the official Next.js docs recommend either the wizard or manual setup and, for App Router + Turbopack, expect `instrumentation-client.ts`, `sentry.server.config.ts`, and `sentry.edge.config.ts`. If you enable Session Replay later, note that Sentry says Replay masks DOM text, images, and user input by default, which is important for a privacy-sensitive product. ²¹

For **PostHog**, install only the pieces you need first. PostHog recommends a reverse proxy because trackers are less likely to be blocked, but it also warns that routing analytics and especially session recordings through a Next.js/Vercel proxy can increase Edge Request and transfer costs. For your product, start with event analytics and feature flags only; leave session recording off until privacy and cost controls are mature. ²²

For **CSP**, Next.js has an official guide and recommends using CSP to mitigate XSS, clickjacking, and code injection threats. That is especially important because you will accept user-generated text, images, and possibly video. ²³

Tests:

```

pnpm vitest run
pnpm exec playwright test tests/reporting.spec.ts tests/admin.spec.ts
pnpm lint
pnpm exec tsc --noEmit

```

Agent operating procedure and PR discipline

The single highest leverage move is to make the agent obey **project rules before code generation**. Next.js now explicitly supports this workflow through `AGENTS.md`, and the docs explain that `create-next-app` generates `AGENTS.md` plus `CLAUDE.md` automatically and that agents should consult the bundled version-matched docs in `node_modules/next/dist/docs/` first. ²⁴

Use this **session-opening prompt** at the start of every agent session:

Read AGENTS.md and any project-specific rules before changing code.

Operating mode for this repo:

- Work only on the requested sprint.
- Do not refactor unrelated code.
- Use Next.js App Router patterns from bundled docs.
- Keep Prisma access in DAL modules only.
- Validate all external input with Zod.
- Add or update tests for every behavior change.
- If schema changes, include Prisma migration and rollback notes.
- If security assumptions change, call them out explicitly.
- Stop after producing a reviewable diff and test results.

Use this **iterative code review prompt** after each implementation pass:

Review your own diff as if you were the maintainer.

Check:

- leaked server secrets into client bundle
- direct Prisma usage outside DAL
- missing auth or RLS guards
- missing token expiry / replay protection
- incorrect cache behavior for user-specific pages
- unsafe HTML rendering
- missing loading / error states
- missing lint, typecheck, or tests

Then make one cleanup pass only for issues found above.

Return:

1. summary of issues found
2. files changed
3. exact commands run
4. remaining risks

Use this **security pass prompt** before merging any feature that touches public endpoints, uploads, or payments:

Perform a focused security review of the current branch.

Threats to check:

- IDOR / horizontal privilege escalation
- expired or replayed public reply tokens
- forged forward tokens
- webhook forgery

- mass assignment in route handlers
- SSRF via remote media URLs
- oversized upload abuse
- missing moderation gating
- missing audit log for admin actions
- CSP gaps
- PII leaking into analytics or error reporting

Make only the minimum safe fixes needed for this branch.

Use this **commit-friendly PR prompt** to control diff size and keep your AI-generated changes reviewable:

Prepare this work as a commit-friendly PR.

Rules:

- one vertical slice only
- no unrelated renames or formatting churn
- keep net diff small unless generated types/migrations require more
- include tests in same PR
- include migration in same PR if schema changed
- add PR summary with:
 - scope
 - files
 - risk
 - rollback
 - test evidence

Before finishing, propose a commit message and a PR title.

A practical PR hygiene standard for solo work is: **one concern per PR, schema and app changes together, green typecheck/lint/tests, migration notes, and explicit rollback notes**. That is not a vendor rule; it is how you keep an AI-assisted repo from becoming unmaintainable.

CI, deployment, launch checklist, and risk plan

For deployment, use Vercel CLI for both local parity and CI. Vercel documents `vercel env`, `vercel build`, and `vercel deploy --prebuilt`, and recommends pulling environment/project settings before local builds. Prisma recommends running `prisma migrate deploy` in CI/CD rather than from a local machine, and specifically notes that some platforms such as Vercel may prune dev dependencies during build, which is one reason to keep migrations in a separate pipeline step. ²⁵

A practical deployment sequence is:

```

vercel link
vercel env add DATABASE_URL production
vercel env add OPENAI_API_KEY production
vercel env add STRIPE_SECRET_KEY production
vercel env add STRIPE_WEBHOOK_SECRET production
vercel env add RESEND_API_KEY production
vercel env add NEXT_PUBLIC_POSTHOG_KEY production
vercel env add NEXT_PUBLIC_SENTRY_DSN production

vercel pull --yes
pnpm install
pnpm exec tsc --noEmit
pnpm vitest run
pnpm exec playwright test
vercel build --prod
vercel deploy --prebuilt --prod

```

For CI, wire migrations into a separate job before deploy:

```

name: ci-deploy

on:
  push:
    branches: [main]

jobs:
  migrate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: pnpm install --frozen-lockfile
      - run: npx prisma migrate deploy
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }

  test-and-build:
    runs-on: ubuntu-latest
    needs: migrate
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22

```

```

- run: pnpm install --frozen-lockfile
- run: pnpm exec tsc --noEmit
- run: pnpm vitest run
- run: pnpm exec playwright install --with-deps
- run: pnpm exec playwright test
- run: vercel build --prod --yes
env:
  VERCEL_TOKEN: ${ secrets.VERCEL_TOKEN }
- run: vercel deploy --prebuilt --prod --token=${ secrets.VERCEL_TOKEN }

```

The launch checklist below is the minimum I would require before public release:

Area	Must-have launch checks
Legal and terms	Terms of service, privacy policy, community rules, explicit 18+ only statement, moderation appeal/contact path
Privacy	No contact scraping, private-by-default request visibility, hashed public tokens, data retention policy, export/delete workflow
Age gating	DOB/self-attestation gate, under-18 block, age check on sign-up and first paid action
Payments	Stripe live keys, verified webhook secret, tested success/cancel flows, idempotent webhook handling, refund playbook
Moderation	Report buttons on requests/replies, admin queue, audit log, block/suspend flow, escalation policy for illegal content
Security	CSP, secure cookies, server-only DAL, RLS policies tested, signed public tokens, rate limiting, upload MIME/size limits
Observability	Sentry server/client init, low-noise PostHog events, no sensitive payloads in logs, deploy health check
Product analytics	Track request created, reply submitted, request solved, forward used, payment completed, report submitted
Ops	Seed admin account, backup/restore tested, migration procedure documented, rollback steps documented
Growth	Installable PWA working, basic SEO pages, share preview images, waitlist/contact capture

This checklist is grounded in the legal and platform guidance most relevant to your app: GDPR privacy-by-default, DSA reporting and appeals, and the extra safeguarding complexity that comes with serving minors. Since you explicitly want an 18+ product, that policy simplifies the system, but it does not remove your need for moderation, reporting, and privacy design. ²⁶

The main abuse and privacy risks are predictable. **Stalking and doxxing** are the most serious product misuse risks, so block requests about tracking private individuals, ex-partners, or non-public locations

without consent. **Scams and fabricated evidence** are the second risk, so keep evidence provenance, moderation status, and audit history separate from user-facing claims, and never label something “verified” unless the claim semantics are precise. **Token abuse** is the biggest technical risk in a reply-without-account system, so use hashed single-purpose expiring tokens, constant-time comparison, rate limits, and replay invalidation. **Analytics/privacy leakage** is the biggest operational risk, so keep PostHog lean and avoid session recording by default. **Marketplace-compliance creep** is the payments risk, so keep v1 monetization platform-directed and defer helper payouts until you are ready for Connect and payout compliance. These are design recommendations informed by the official docs on moderation, privacy-by-default, analytics proxying, and multi-party payments. ²⁷

The shortest path to a strong first public build is therefore:

1. Foundation and security boundary
2. Text-only request and reply loop
3. Media evidence with moderation gate
4. Forwarding and notifications
5. Payments and AI helper layer
6. Admin, analytics, PWA, and deployment hardening

If you keep the agent constrained to that order, you will get a codebase that is much closer to production quality than if you ask it to “build the whole app” in one pass.

¹ ¹² **Guides: PWAs | Next.js**

<https://nextjs.org/docs/app/guides/progressive-web-apps>

² ⁵ **PostgreSQL: Documentation: 18: 5.9. Row Security Policies**

<https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

³ ²⁴ **Guides: AI Coding Agents | Next.js**

<https://nextjs.org/docs/app/guides/ai-agents>

⁴ **Guides: Data Security | Next.js**

<https://nextjs.org/docs/app/guides/data-security>

⁶ ¹⁵ **Next.js App Router Setup - UploadThing Docs**

<https://docs.uploadthing.com/getting-started/appdir>

⁷ ¹⁸ **docs.stripe.com**

<https://docs.stripe.com/payments/checkout>

⁸ **Text generation | OpenAI API**

<https://developers.openai.com/api/docs/guides/text>

⁹ ¹⁷ ²⁶ **Art. 25 GDPR – Data protection by design and by default**

https://gdpr-info.eu/art-25-gdpr/?utm_source=chatgpt.com

¹⁰ **Deploying database changes with Prisma Migrate | Prisma Documentation**

<https://www.prisma.io/docs/orm/prisma-client/deployment/deploy-database-changes-with-prisma-migrate>

¹¹ **CLI: create-next-app | Next.js**

<https://nextjs.org/docs/app/api-reference/cli/create-next-app>

13 Environment variables

<https://vercel.com/docs/environment-variables>

14 Getting Started: Route Handlers | Next.js

<https://nextjs.org/docs/app/getting-started/route-handlers>

16 Presigned URLs · Cloudflare R2 docs

<https://developers.cloudflare.com/r2/api/s3/presigned-urls/>

19 Platforms and marketplaces with Stripe Connect

https://docs.stripe.com/connect?utm_source=chatgpt.com

20 The Digital Services Act | Shaping Europe's digital future

https://digital-strategy.ec.europa.eu/en/policies/digital-services-act?utm_source=chatgpt.com

21 Manual Setup | Sentry for Next.js

<https://docs.sentry.io/platforms/javascript/guides/nextjs/manual-setup/>

22 Web analytics installation - Docs - PostHog

<https://posthog.com/docs/web-analytics/installation>

23 Guides: Content Security Policy

https://nextjs.org/docs/app/guides/content-security-policy?utm_source=chatgpt.com

25 vercel env

<https://vercel.com/docs/cli/env>

27 Moderation | OpenAI API

<https://developers.openai.com/api/docs/guides/moderation>